Unit V

Dr. Ayush K. Agrawal

UPES Dehradun

Introduction to Optimization
&
Linear Programming

Foundation of decision-making and computational optimization

What is Optimization?

Optimization:

- Process of finding the best solution among many feasible solutions
- Objective: Maximize or Minimize a function

Examples:

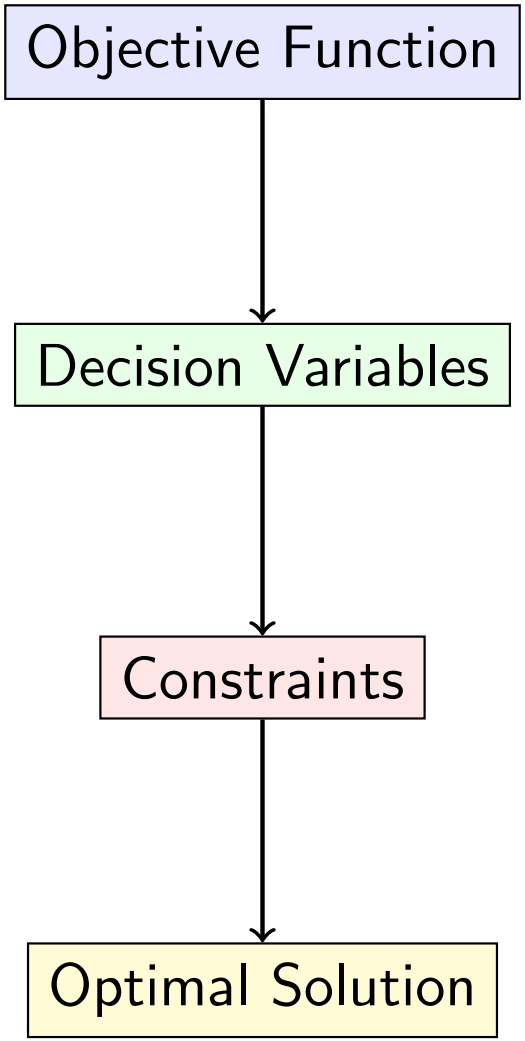
- Minimize cost of production
- Maximize profit
- Optimize robot path planning

Real-World Motivation

Applications of Optimization

- Robotics: Path planning and motion optimization
- Logistics: Transportation cost minimization
- Manufacturing: Resource allocation
- Cybersecurity: Task scheduling and load balancing

Optimization Framework



Insight: Optimization connects variables, constraints, and goals.

What is Linear Programming?

Linear Programming (LP):

- Optimization of a linear objective function
- Subject to linear constraints

Mathematical Form:

$$\text{Maximize } Z = c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

$$\text{Subject to constraints and } x_i \geq 0$$

Components of Linear Programming

- **Decision Variables:** Unknown quantities (x_1, x_2)
- **Objective Function:** Function to optimize
- **Constraints:** Resource limitations
- **Feasible Region:** Set of valid solutions

Real-Life LP Example

Problem:

- Two products with profits INR 40 and INR 30
- Limited resources available

Formulation:

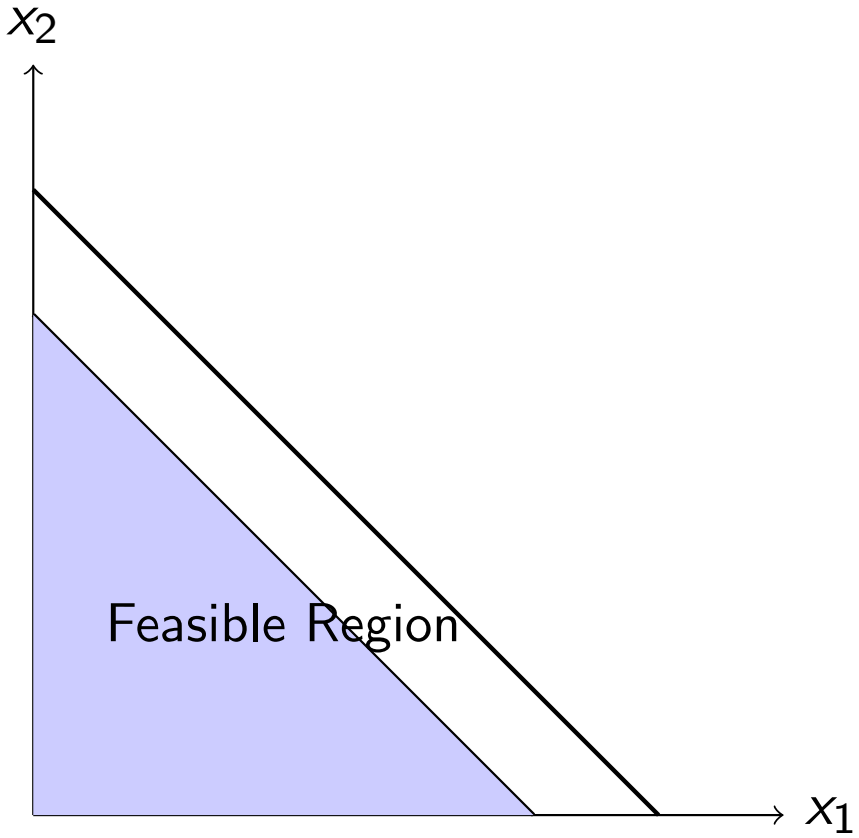
$$\text{Maximize } Z = 40x_1 + 30x_2$$

$$2x_1 + x_2 \leq 100$$

$$x_1 + x_2 \leq 80$$

$$x_1, x_2 \geq 0$$

Graphical Insight of LP



Key Idea: All valid solutions lie inside the shaded region.

Why Linear Programming Matters

- Helps in structured decision-making
- Widely used in:
  - Robotics and automation
  - Supply chain optimization
  - AI and data systems
- Basis for advanced optimization techniques

LP in Computational Complexity

- Linear Programming is efficiently solvable
- Polynomial-time algorithms exist
- Contrast with NP-complete problems (upcoming lectures)

Bridge to Exam: LP vs NP-hard problems in real-world decision-making

Summary

- Optimization is central to engineering problems
- LP provides mathematical structure
- Feasible region defines valid solutions
- Foundation for complexity theory and algorithms

Mathematical Formulation of Linear Programming

From real-world problems to mathematical models

Why Mathematical Formulation?

- Real-world problems are often unstructured
- Mathematical formulation converts them into solvable models
- Essential for applying algorithms like Simplex

Goal:

- Define variables
- Construct objective function
- Translate constraints mathematically

General Structure of LP

$$\text{Maximize or Minimize } Z = c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

Subject to:

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \leq b_1$$

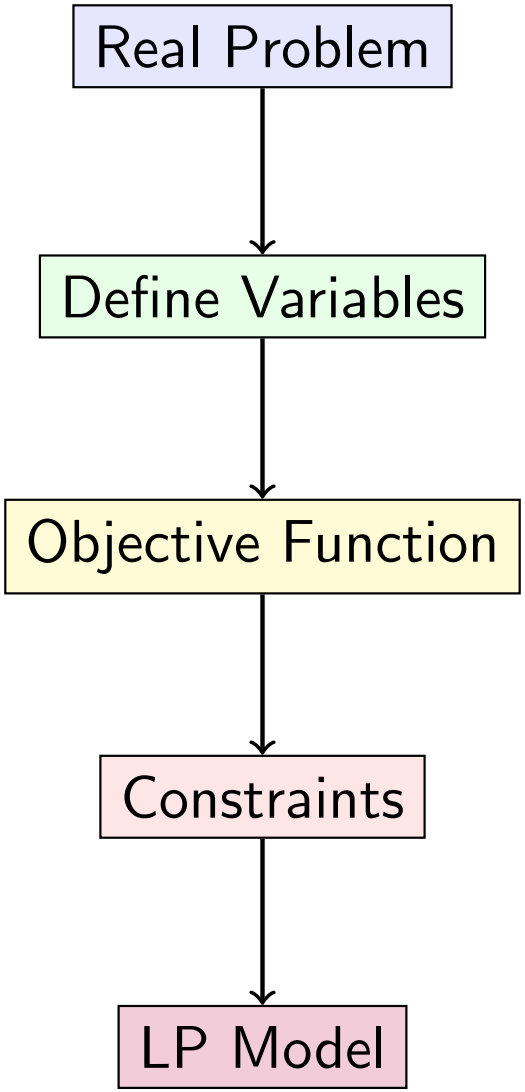
$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \geq b_2$$

$$x_i \geq 0$$

Key Components Explained

- **Decision Variables:** Unknowns $(x_1, x_2, ..., x_n)$
- **Objective Function:** Linear combination to optimize
- **Constraints:** Linear inequalities or equalities
- **Non-negativity:** $x_i \geq 0$

Flow of Problem Formulation



Example: Production Problem

Given:

- Two products A and B
- Profit: INR 50 and INR 40 per unit
- Resource constraints:

$$3x_1 + 2x_2 \leq 120$$

$$x_1 + 2x_2 \leq 80$$

Objective:

$$\text{Maximize } Z = 50x_1 + 40x_2$$

Standard Form of LP

Standard Form Requirements:

- All constraints as equations
- Right-hand side positive
- Variables non-negative

Conversion:

$$3x_1 + 2x_2 + s_1 = 120$$

$$x_1 + 2x_2 + s_2 = 80$$

Slack Variables

Purpose:

- Convert inequality ($\leq$) into equality

Example:

$$x_1 + x_2 \leq 10 \Rightarrow x_1 + x_2 + s = 10$$

Interpretation:

- s represents unused resource

Surplus and Artificial Variables

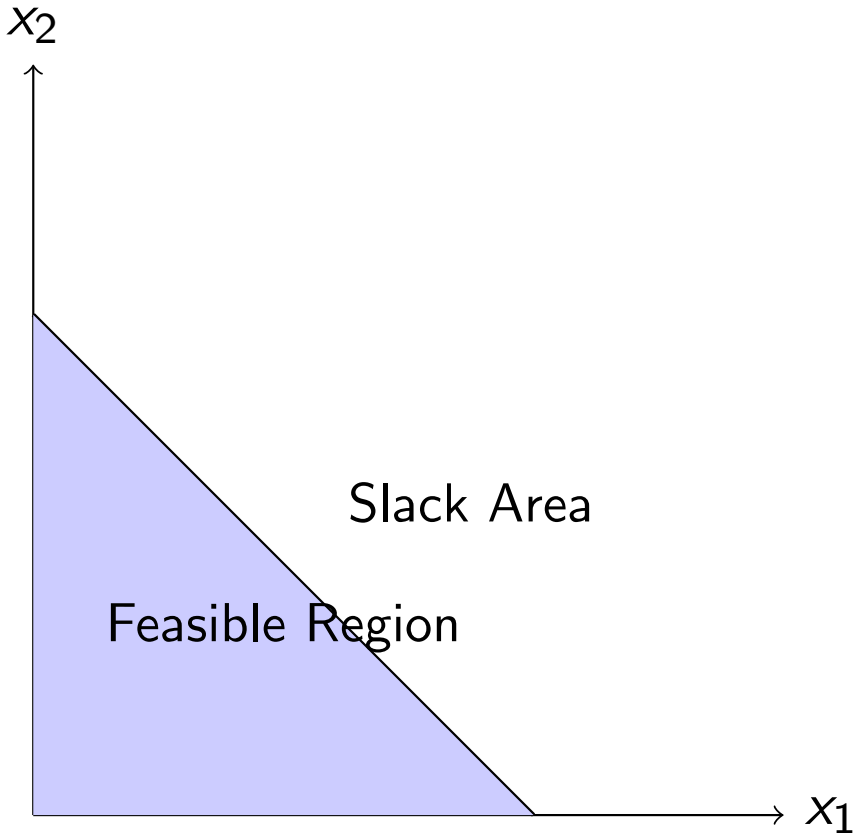
For $\geq$ constraints:

$$x_1 + x_2 \geq 10 \Rightarrow x_1 + x_2 - s = 10$$

Artificial Variables:

- Introduced for feasibility
- Used in Big-M and Two-phase methods

Visual Meaning of Slack Variable



Slack represents unused portion within constraints.

Important Observations

- Every LP problem can be converted to standard form
- Slack variables are crucial for Simplex method
- Proper formulation determines correctness of solution

Link to Next Lecture

- Once LP is formulated:
  - We analyze feasible region geometrically
  - Identify optimal solutions at corner points

Next: Geometry of Feasible Region

Summary

- Real-world problems must be mathematically modeled
- LP consists of objective + constraints
- Slack and surplus variables enable standard form
- Foundation for Simplex algorithm

Geometry of Feasible Region

Understanding Linear Programming through graphical interpretation

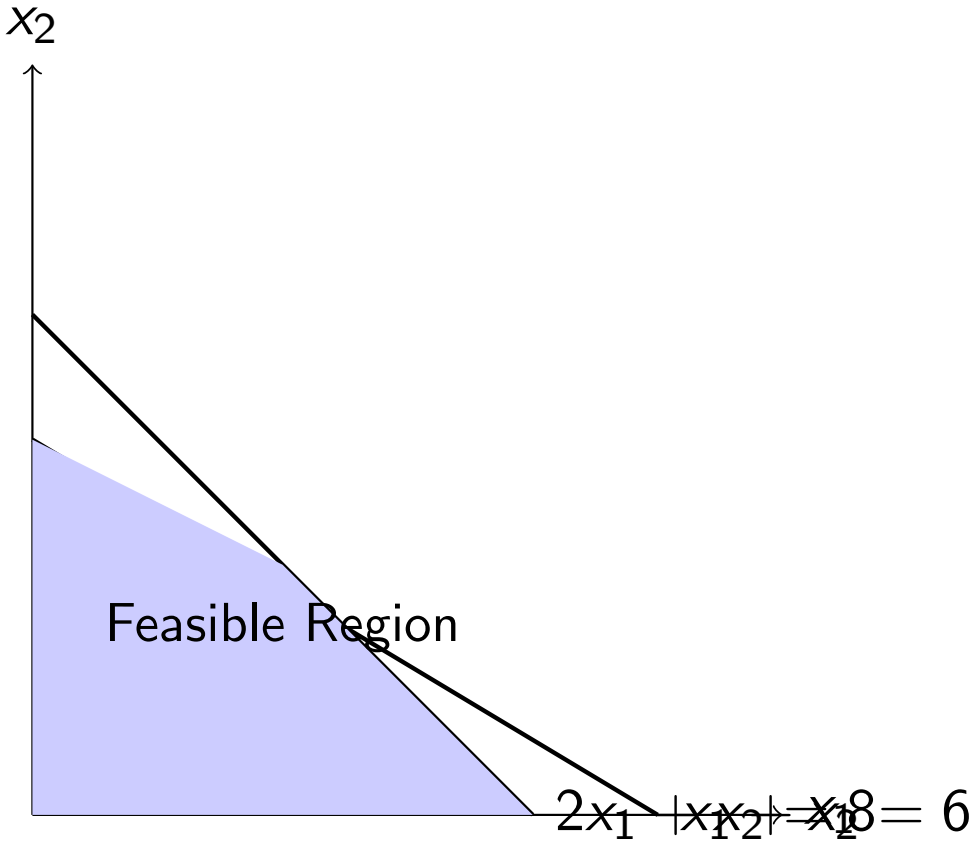
What is Feasible Region?

- Set of all points satisfying all constraints
- Represents all possible valid solutions
- Graphically: Intersection of half-planes

Important:

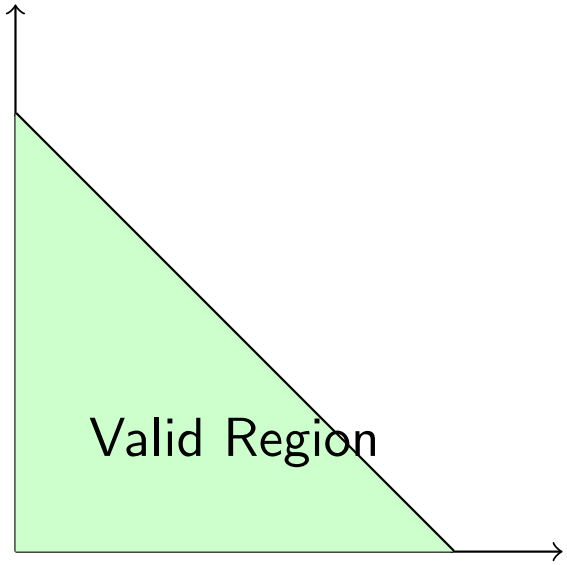
- Always a convex region

Graphical Representation of Constraints



Half-Plane Concept

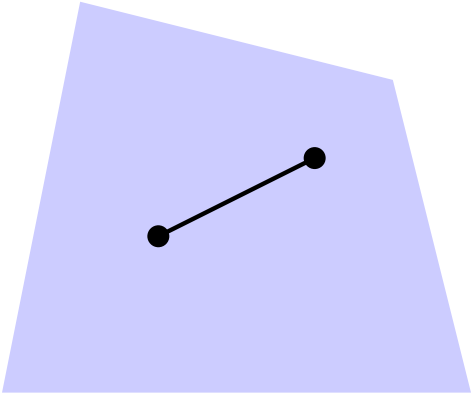
- Each linear inequality divides plane into two halves
- One half satisfies constraint $\rightarrow$ feasible half-plane



Convex Sets

Definition:

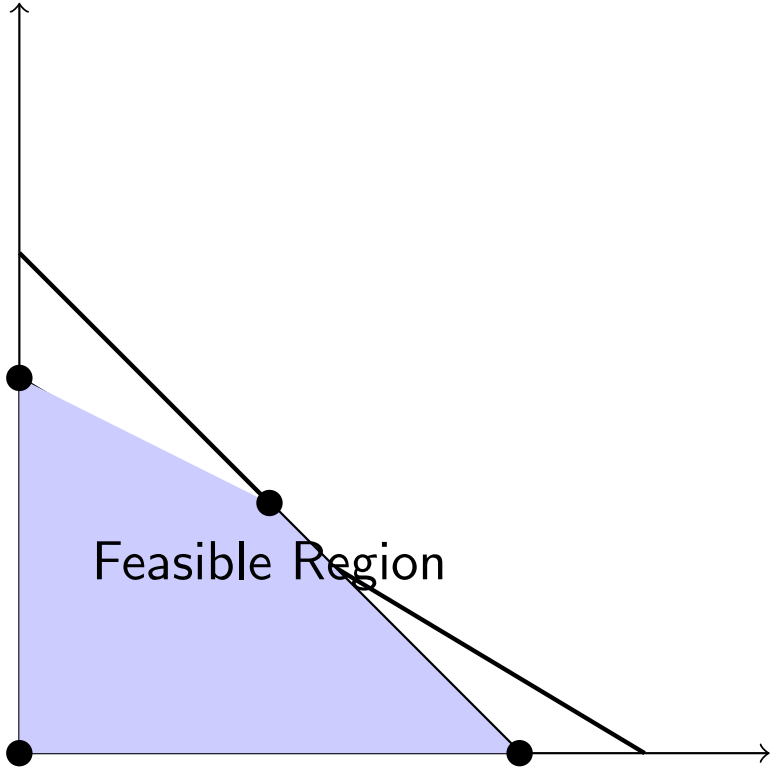
- A set is convex if line segment between any two points lies inside the set



Key Insight: Feasible region is always convex.

Corner Points (Extreme Points)

- Intersection points of constraint boundaries
- Also called vertices of feasible region



Corner Point Theorem

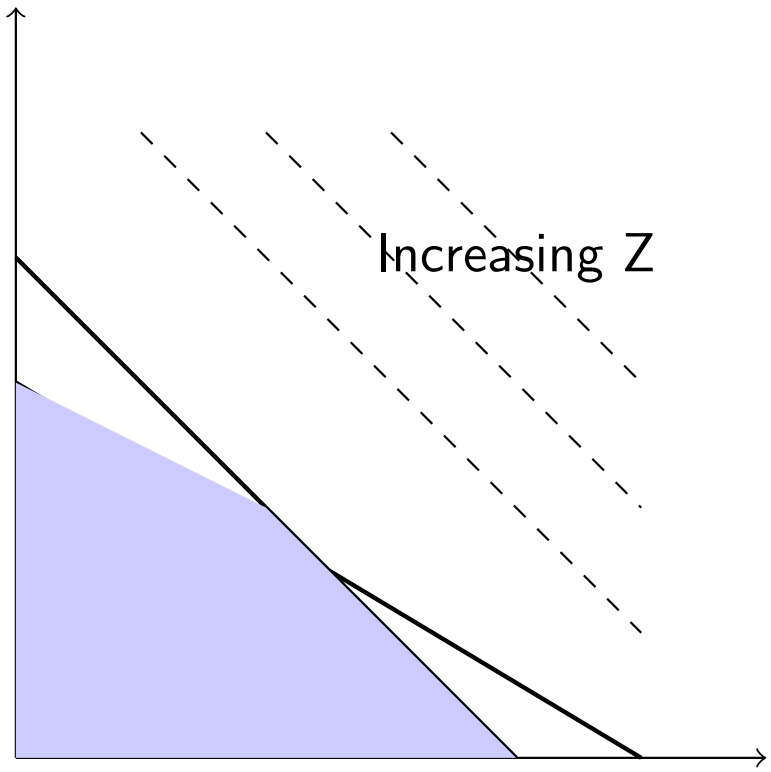
Statement:

- Optimal solution of LP occurs at a corner point

Implication:

- Instead of infinite points $\rightarrow$ check finite vertices
- Foundation of Simplex Algorithm

Objective Function Lines

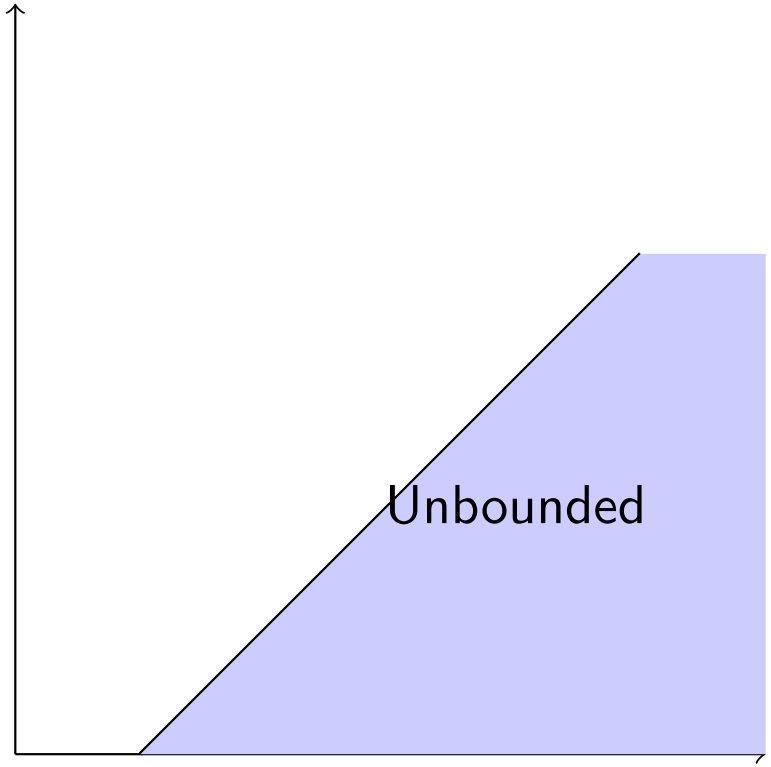


Objective function shifts parallelly to reach optimum.

Types of Solutions

- **Unique Solution:** Single optimal point
- **Multiple Solutions:** Infinite optimal points
- **Unbounded Solution:** Objective grows indefinitely
- **Infeasible:** No common region

Unbounded Region Example



Exam-Oriented Insight

For Answer Writing:

- Define feasible region
- Explain half-planes
- Mention convexity
- State corner point theorem
- Explain objective function movement

This directly answers:
"Geometry of feasible regions in Linear Programming"

Link to Simplex Algorithm

- Geometry shows optimal solution at vertices
- Simplex systematically moves between vertices

Next Lecture: Simplex Algorithm (Concept + Working)

Summary

- Feasible region is intersection of constraints
- Always convex
- Optimal solution lies at corner points
- Geometry forms basis of Simplex method

Simplex Algorithm

Concept, Working Principle, and Geometric Insight

Why Do We Need Simplex?

- Graphical method works only for 2 variables
- Real-world problems involve many variables
- Need a systematic method to reach optimal solution

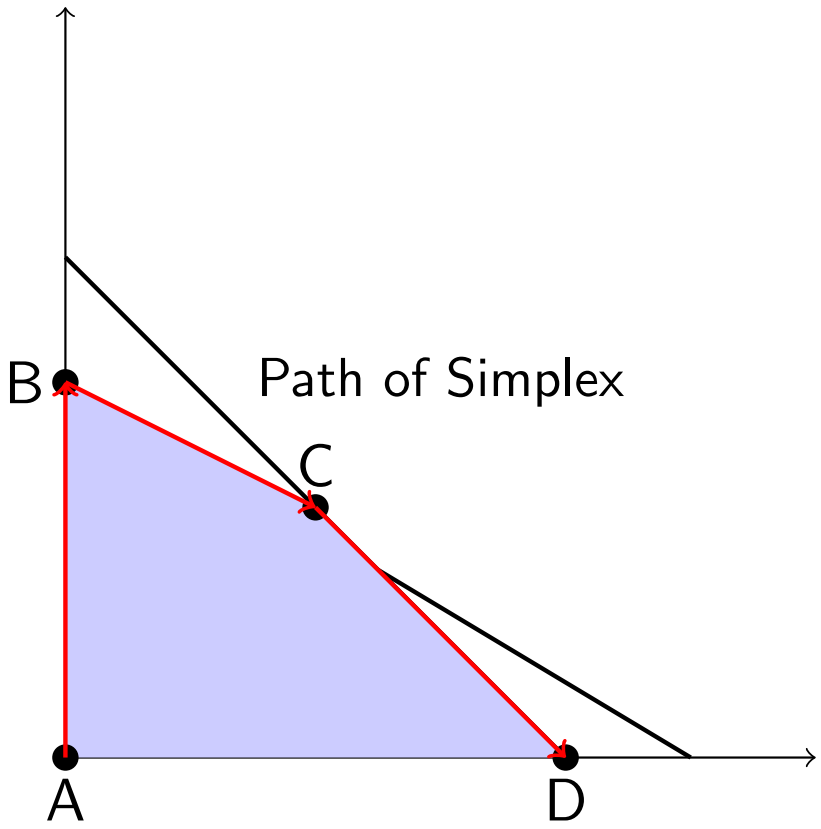
Solution:

- Simplex Algorithm

Basic Idea of Simplex

- Start from an initial corner point
- Move to adjacent corner points
- Improve objective function at each step
- Stop when no further improvement is possible

Geometric Interpretation



Simplex moves from one vertex to another improving solution.

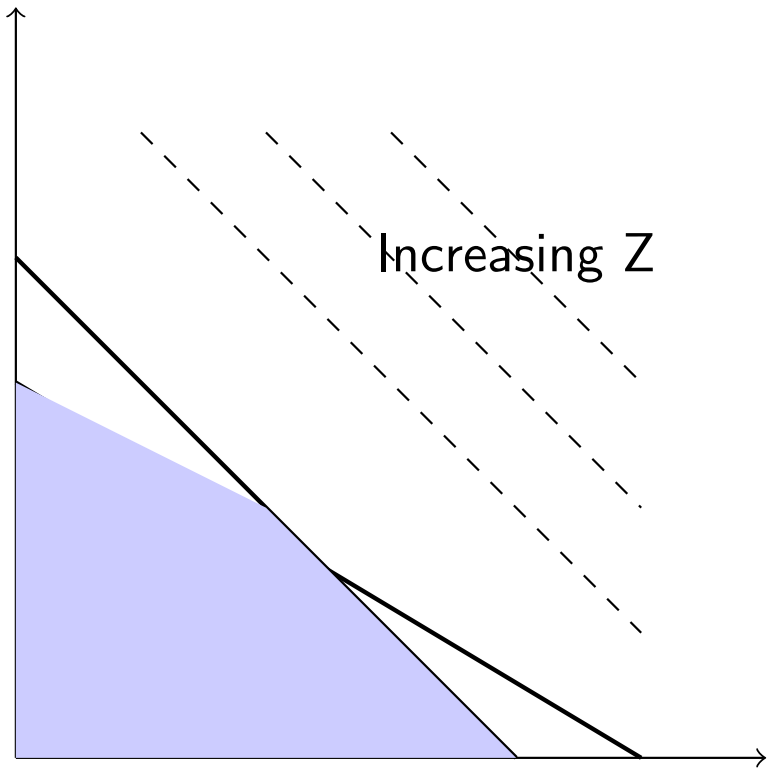
Key Terminology

- **Basic Variables:** Variables in solution
- **Non-basic Variables:** Set to zero
- **Feasible Solution:** Satisfies all constraints
- **Optimal Solution:** Best feasible solution

How Simplex Works (High-Level)

1. Convert LP to standard form
2. Start with initial basic feasible solution
3. Select entering variable
4. Select leaving variable
5. Perform pivot operation
6. Repeat until optimality

Improvement of Objective Function



Each step improves objective value.

Stopping Condition

- No further improvement possible
- All coefficients in objective row are non-negative (for maximization)

Meaning:

- Current solution is optimal

Why Simplex is Efficient

- Does not check all possible solutions
- Moves intelligently along vertices
- Very fast in practice

Note: Worst-case exponential, but performs well in real systems.

Connection to Exam Question

To explain Simplex in exam:

- Mention vertex-to-vertex movement
- Explain improvement of objective function
- Describe stopping condition
- Link with feasible region geometry

This directly answers: "Working of the Simplex Algorithm"

Link to Next Lecture

- Now we convert concept into computation
- Use tableau method for calculations

Next: Simplex Algorithm (Tableau Method)

Summary

- Simplex moves across vertices
- Improves objective function step-by-step
- Stops at optimal solution
- Practical and widely used algorithm

Simplex Algorithm

Tableau Method & Numerical Solution

Problem Statement

Maximize:

$$Z = 3x_1 + 2x_2$$

Subject to:

$$2x_1 + x_2 \leq 18$$

$$2x_1 + 3x_2 \leq 42$$

$$3x_1 + x_2 \leq 24$$

$$x_1, x_2 \geq 0$$

Convert to Standard Form

Introduce slack variables:

$$2x_1 + x_2 + s_1 = 18$$

$$2x_1 + 3x_2 + s_2 = 42$$

$$3x_1 + x_2 + s_3 = 24$$

Initial basic variables: s_1, s_2, s_3

Initial Simplex Tableau

Basis	x_1	x_2	s_1	s_2	s_3	RHS
s_1	2	1	1	0	0	18
s_2	2	3	0	1	0	42
s_3	3	1	0	0	1	24
Z	-3	-2	0	0	0	0

Step 1: Entering Variable

- Choose most negative value in Z-row

$$-3 \Rightarrow x_1 \text{ enters}$$

Step 2: Leaving Variable

Compute ratios:

$$\frac{18}{2} = 9, \quad \frac{42}{2} = 21, \quad \frac{24}{3} = 8$$

Minimum ratio = 8

$$s_3 \text{ leaves}$$

Pivot element = 3

Pivot Operation

- Make pivot element $= 1$
- Make all other elements in column $= 0$

$$\boxed{\text{Row operations transform tableau}}$$

Updated Tableau

Basis	x_1	x_2	s_1	s_2	s_3	RHS
s_1	0	$\frac{1}{3}$	1	0	$-\frac{2}{3}$	2
s_2	0	$\frac{7}{3}$	0	1	$-\frac{2}{3}$	26
x_1	1	$\frac{1}{3}$	0	0	$\frac{1}{3}$	8
Z	0	-1	0	0	1	24

Next Iteration

- Most negative value in Z-row = -1
- x_2 enters

Compute ratios:

$$\frac{2}{1/3} = 6, \quad \frac{26}{7/3} \approx 11.14$$

s_1 leaves

Final Tableau

Basis	x_1	x_2	RHS
x_2	0	1	6
x_1	1	0	6
Z	0	0	30

Optimal Solution

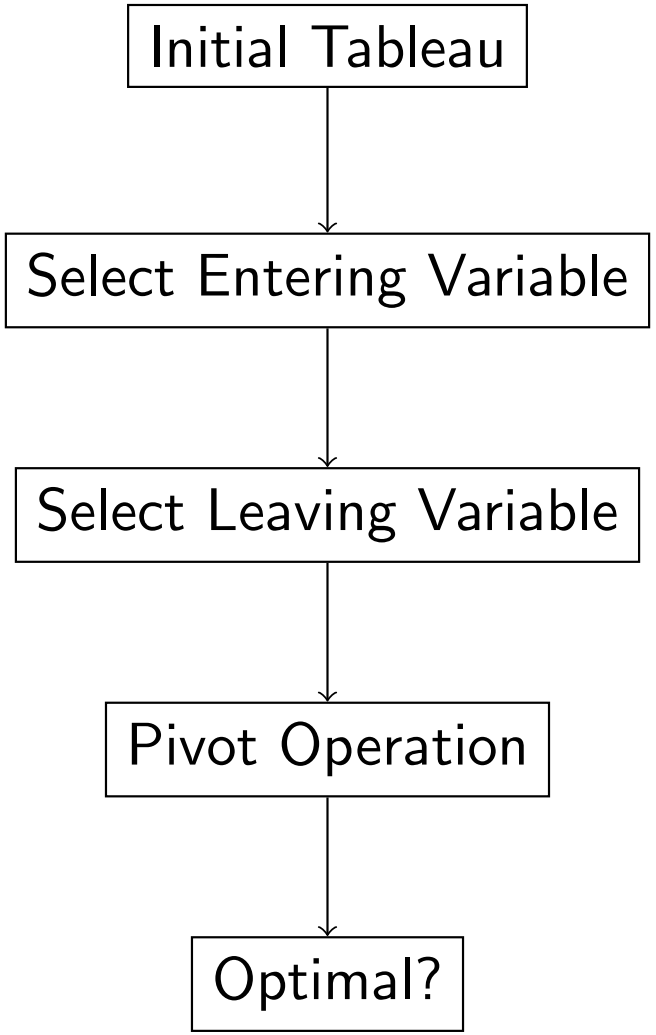
$$x_1 = 6, \quad x_2 = 6$$

$$Z_{max} = 30$$

Conclusion:

- Optimal solution reached
- No negative values in Z-row

Simplex Flow Summary



Exam-Oriented Tips

- Always convert to standard form
- Clearly identify entering and leaving variables
- Show ratio calculations
- Perform pivot carefully
- Final answer must include optimal values

Summary

- Simplex uses tableau for computation
- Iterative improvement process
- Stops at optimal solution
- Most important for numerical exam problems

Computational Complexity

Understanding P, NP and Problem Classification

Why Study Complexity?

- Not all problems are easy to solve
- Some problems take too long (intractable)
- Helps in choosing correct algorithmic approach

Key Question:

- Can a problem be solved efficiently?

Time Complexity Recap

- Measures how runtime grows with input size
- Polynomial time: $O(n), O(n^2), O(n^3)$
- Exponential time: $O(2^n), O(n!)$

Observation:

- Polynomial $\rightarrow$ Efficient
- Exponential $\rightarrow$ Infeasible for large inputs

Class P (Polynomial Time)

Definition:

- Problems solvable in polynomial time

Examples:

- Sorting (Merge Sort)
- Shortest Path (Dijkstra)
- Linear Programming

Class NP (Nondeterministic Polynomial Time)

Definition:

- Solutions can be verified in polynomial time

Key Idea:

- Easy to check, but not necessarily easy to solve

Example:

- Sudoku verification
- Boolean satisfiability (SAT)

P vs NP Intuition

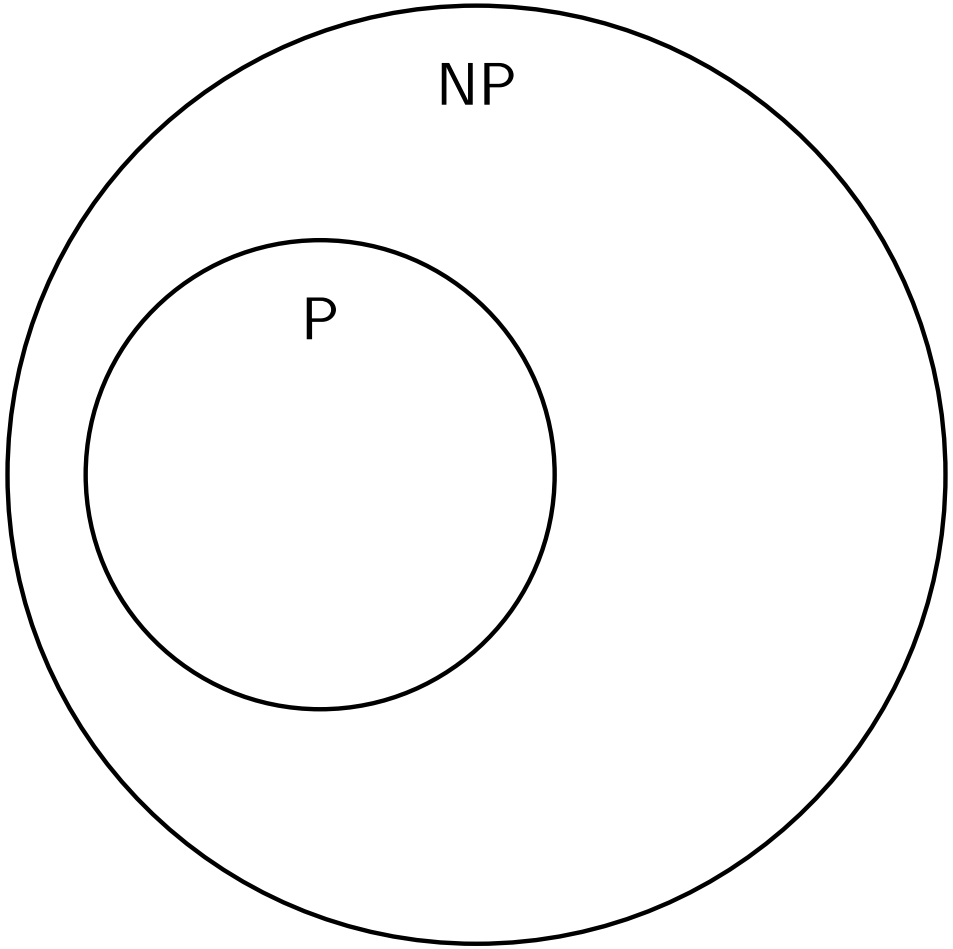
- P: Easy to solve and verify
- NP: Hard to solve, easy to verify

Classic Question:

$$\text{Is } P = NP?$$

Status: Still an open problem

Graphical Representation



P is a subset of NP

Decision Problems

Definition:

- Problems with YES/NO answers

Example:

- Is there a path of length$\leq k$?
- Does a solution exist?

Note:

- Complexity theory focuses on decision problems

Verification Concept

Verifier:

- Algorithm that checks solution quickly

Example:

- Given solution to SAT $\rightarrow$ check in polynomial time

Insight:

- Verification is easier than solving

Real-World Interpretation

- Scheduling problems $\rightarrow$ Hard to solve optimally
- Optimization problems $\rightarrow$ Often NP
- Verification of solutions $\rightarrow$ Usually easy

Engineering Insight:

- Leads to approximation and heuristics

Exam-Oriented Insight

To write in exam:

- Define P and NP clearly
- Explain polynomial vs exponential
- Mention verification concept
- Draw $P \subseteq NP$ diagram

This supports:
"Classification of problems based on tractability"

Link to Next Lecture

- Now we extend NP to:
  - NP-Complete problems
  - NP-Hard problems

Next: NP-Completeness and NP-Hardness

Summary

- Complexity classifies problems by difficulty
- P: Efficiently solvable
- NP: Efficiently verifiable
- Foundation for NP-completeness

NP-Complete & NP-Hard Problems

Understanding Computational Intractability

Recap: P vs NP

- **P**: Solvable in polynomial time
- **NP**: Verifiable in polynomial time

Next Question:

- Which problems are the hardest in NP?

NP-Complete Problems

Definition:

- Problems that are:
  - In NP
  - As hard as any problem in NP

Meaning:

- If one NP-complete problem is solved efficiently $\rightarrow$ all NP problems are solved efficiently

NP-Hard Problems

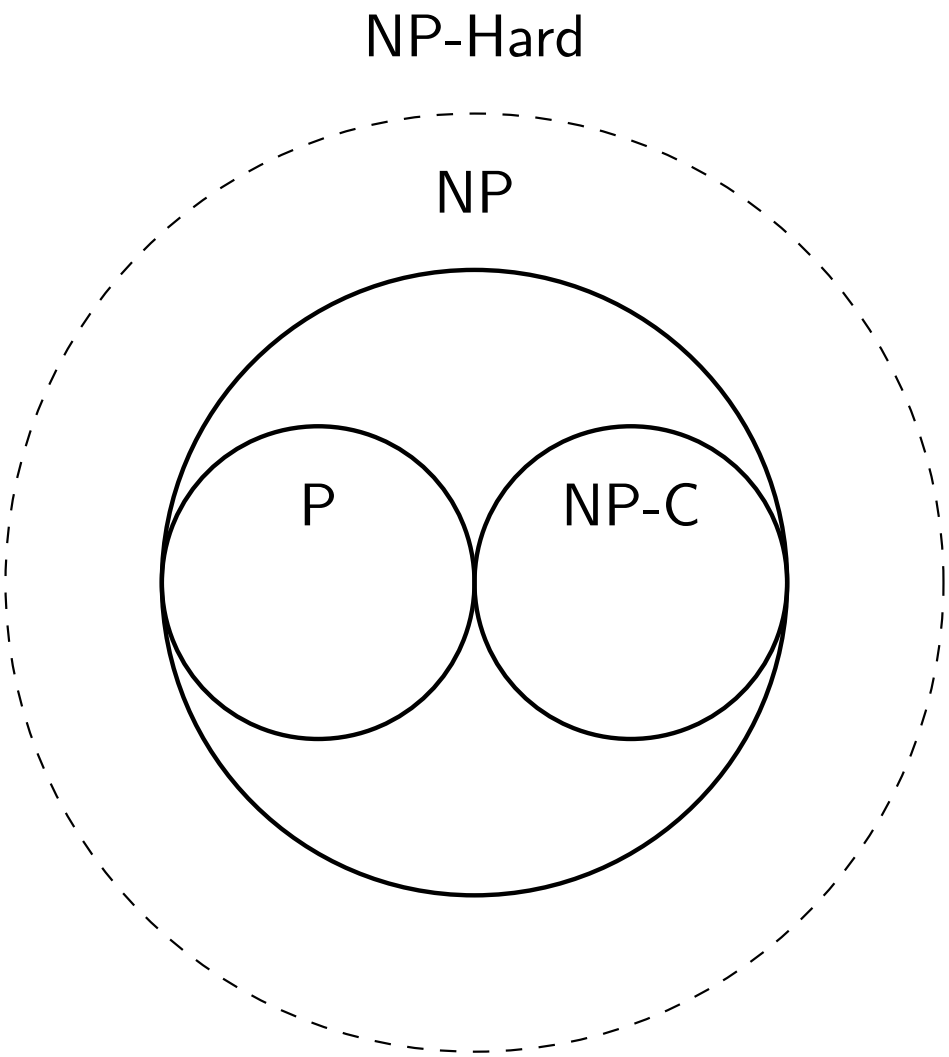
Definition:

- At least as hard as NP-complete problems
- Not necessarily in NP

Key Insight:

- May not even have polynomial-time verification

Relationship Between Classes



NP-Complete lies inside NP, NP-Hard may lie outside NP

Examples of NP-Complete Problems

- Boolean Satisfiability (SAT)
- 3-SAT
- Clique Problem
- Vertex Cover
- Hamiltonian Cycle

Examples of NP-Hard Problems

- Traveling Salesman Problem (Optimization version)
- Halting Problem
- Scheduling optimization problems

Characteristics of NP-Hard Problems

- No known polynomial-time solution
- Computationally expensive
- Often solved using:
  - Approximation algorithms
  - Heuristics
  - Randomized approaches

Exam Point: Always mention these properties

Why NP-Complete Problems Matter

- Represent hardest problems in NP
- Used to classify difficulty of problems
- Basis for computational complexity theory

Real-World Relevance

- Network design
- Resource scheduling
- Cybersecurity problems
- Robotics path optimization

Insight:

- Many real-world problems are NP-hard

Comparison Summary

- P: Easy problems
- NP: Verifiable problems
- NP-Complete: Hardest in NP
- NP-Hard: Even harder (may not be in NP)

Exam-Oriented Answer Structure

To answer NP-hard/NP-complete:

- Define NP
- Define NP-Complete
- Define NP-Hard
- Draw diagram
- Give examples
- Mention real-world relevance

Directly answers: "Characteristics of NP-hard problems"

Link to Next Lecture

- How do we prove a problem is NP-complete?
- Use reduction techniques

Next: NP-Completeness Proofs and Reductions

Summary

- NP-Complete = hardest problems in NP
- NP-Hard = at least as hard as NP-complete
- Important for understanding intractability
- Widely relevant in real-world systems

NP-Completeness Proof

Reduction Technique and Methodology

Why Do We Need Reductions?

- Directly proving a problem is NP-complete is difficult
- Use known NP-complete problems
- Transform one problem into another

Idea:

- If A reduces to B $\rightarrow$ B is at least as hard as A

What is Reduction?

Definition:

- Transform an instance of problem A into problem B
- Such that solution of B solves A

$$A \leq_p B$$

Meaning:

- B is at least as hard as A

Reduction Visualization

$$\boxed{\text{Known NP-Complete (A)}} \xrightarrow{\leq_p} \boxed{\text{New Problem (B)}}$$

If A reduces to B $\rightarrow$ solving B solves A

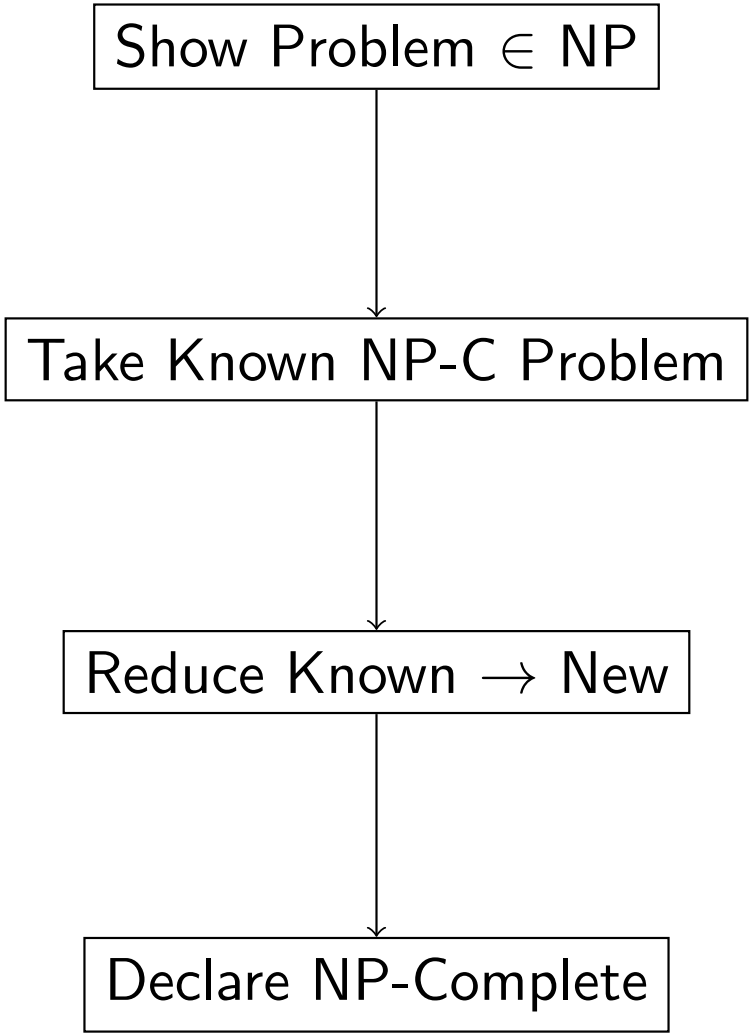
Steps to Prove NP-Completeness

1. Show problem is in NP
2. Take a known NP-complete problem
3. Reduce it to given problem

Conclusion:

- Problem is NP-complete

Flow of NP-Completeness Proof



Important Note on Direction

Correct Reduction:

$$\text{Known NP-C} \rightarrow \text{New Problem}$$

Wrong Direction:

$$\text{New Problem} \rightarrow \text{Known}$$

Key Idea:

- Always reduce from hard problem to new problem

Classic Example: SAT

Boolean Satisfiability (SAT):

- Given Boolean formula
- Check if assignment exists making it TRUE

First NP-Complete Problem

Reduction Chain

$$\boxed{\text{SAT}} \longrightarrow \boxed{\text{3-SAT}} \longrightarrow \boxed{\text{CLIQUE}}$$

Used to prove multiple problems NP-complete

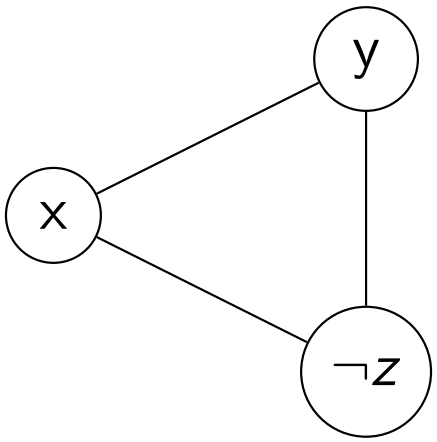
Example Idea: 3-SAT to Clique

- Convert clauses into graph
- Nodes represent literals
- Edges represent compatibility

Goal:

- Find clique of size k

Reduction Concept Visualization



Clique corresponds to satisfying assignment

Why Reductions are Powerful

- Avoid solving from scratch
- Build chain of complexity results
- Standard method in complexity theory

Exam-Oriented Answer Structure

To prove NP-completeness:

- Step 1: Show problem $\in$ NP
- Step 2: Choose known NP-complete problem
- Step 3: Construct reduction
- Step 4: Conclude NP-complete

Important Line: "Since problem A reduces to B in polynomial time, B is NP-complete"

Common Mistakes

- Wrong direction of reduction
- Not proving problem is in NP
- Missing explanation of transformation

Link to Next Lecture

- NP-hard problems are difficult to solve exactly
- Use approximation algorithms

Next: Approximation Algorithms

Summary

- NP-completeness uses reduction
- Always reduce from known NP-complete problem
- SAT is base problem
- Reduction proves computational hardness

Approximation Algorithms

Handling NP-Hard Problems Efficiently

Why Approximation Algorithms?

- Many problems are NP-hard
- Exact solutions take exponential time
- Not feasible for large inputs

Solution:

- Use near-optimal solutions

Idea of Approximation

- Instead of optimal solution $\rightarrow$ find close solution
- Guarantee solution quality

Goal:

- Fast computation
- Acceptable accuracy

Approximation Ratio

Definition:

$$\rho = \frac{\text{Approximate Solution}}{\text{Optimal Solution}}$$

For minimization:

$$\rho \geq 1$$

Example:

- 2-approximation $\rightarrow$ solution at most twice optimal

Types of Approximation Algorithms

- Constant-factor approximation
- Polynomial-time approximation scheme (PTAS)
- Fully polynomial-time approximation scheme (FPTAS)

Vertex Cover Problem

Problem:

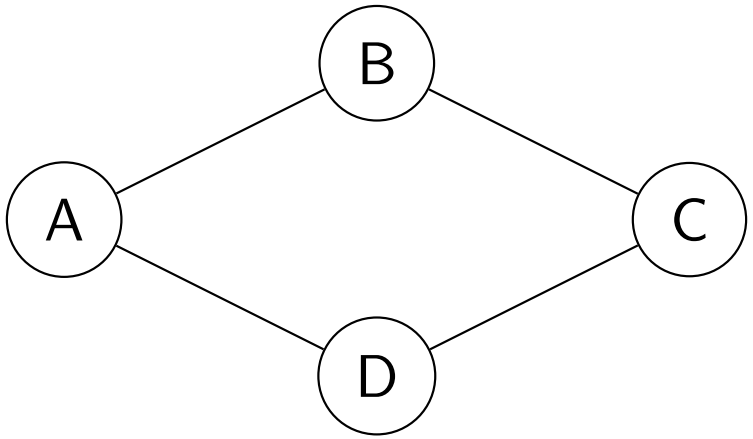
- Select minimum number of vertices
- Such that every edge is covered

NP-Hard Problem

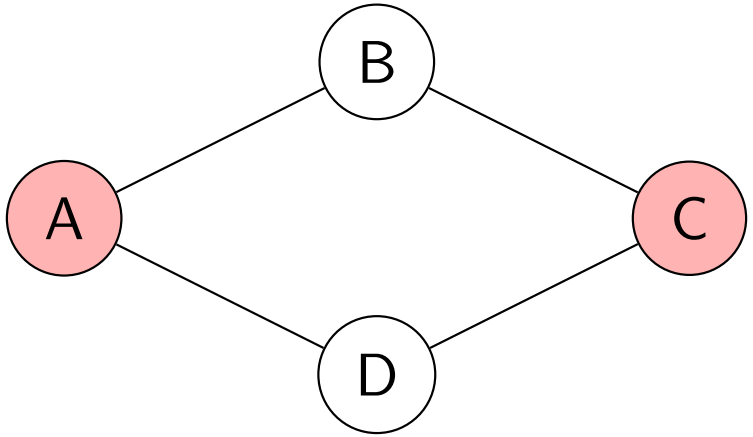
Approximation Algorithm (Vertex Cover)

- Pick any edge (u, v)
- Add both u and v to solution
- Remove all edges incident on u and v
- Repeat until no edges remain

Graph Example



Selected Vertex Cover



Approximate solution may not be optimal but is efficient

Performance Guarantee

- Vertex Cover algorithm is 2-approximation
- Solution size $\leq 2 \times$ optimal

Key Idea:

- Trade-off between accuracy and efficiency

Real-World Importance

- Network design
- Resource allocation
- Scheduling problems
- Robotics optimization

Insight:

- Exact solution not always necessary

Connection to NP-Hard Problems

- NP-hard problems cannot be solved efficiently
- Approximation provides practical solutions

Engineering Approach:

- Use approximation + heuristics

Exam-Oriented Answer Structure

To write in exam:

- Define approximation algorithm
- Explain need (NP-hard problems)
- Define approximation ratio
- Give example (Vertex Cover)
- Mention real-world use

Directly answers: "Importance of approximation algorithms"

Link to Next Lecture

- Another approach for difficult problems:
  - Randomized algorithms

Next: Randomized Algorithms and Advanced Topics

Summary

- Approximation algorithms handle NP-hard problems
- Provide near-optimal solutions
- Efficient and practical
- Widely used in engineering systems

Randomized Algorithms

Practical Techniques for Complex Problems

Why Randomized Algorithms?

- Deterministic algorithms may be slow
- Randomness helps avoid worst-case scenarios
- Often simpler and faster

Idea:

- Use randomness to guide computation

Types of Randomized Algorithms

- **Las Vegas Algorithms**
  - Always correct result
  - Running time varies
- **Monte Carlo Algorithms**
  - Fixed running time
  - Small probability of error

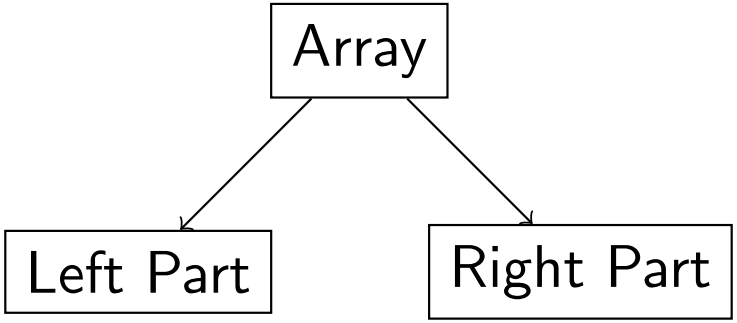
Example: Randomized QuickSort

- Select pivot randomly
- Reduces chance of worst-case

Expected Complexity:

$$O(n \log n)$$

QuickSort Visualization



Random Pivot Splits Data

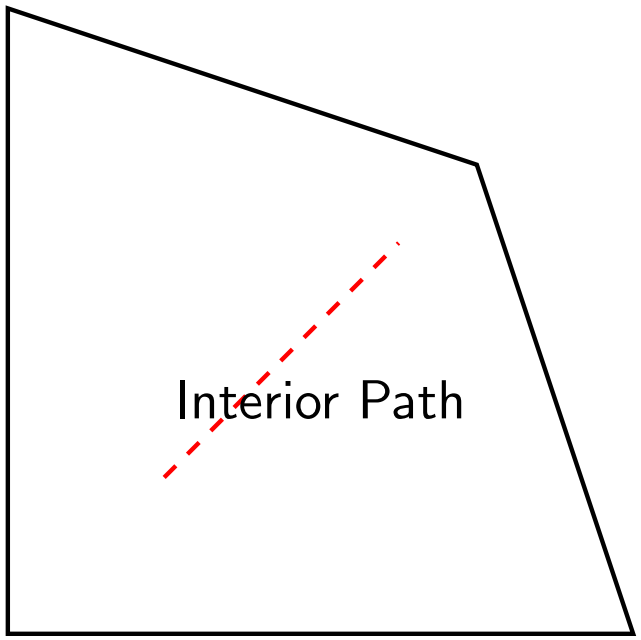
Advantages of Randomized Algorithms

- Simpler implementation
- Good average-case performance
- Avoids worst-case inputs

Interior Point Method

- Alternative to Simplex
- Moves through interior of feasible region
- Polynomial-time algorithm

Interior Point Visualization



Advanced Number Theoretic Algorithms

- Euclidean Algorithm (GCD)
- Modular Arithmetic
- Fast Exponentiation

Fast Exponentiation

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ even} \\ a \cdot (a^{(n-1)/2})^2 & \text{if } n \text{ odd} \end{cases}$$

Time Complexity:

$$O(\log n)$$

Handling Intractable Problems

- NP-hard problems cannot be solved efficiently
- Use:
  - Approximation algorithms
  - Randomized algorithms
  - Heuristics

Full Integration and practice

To answer long questions:

- Linear Programming $\rightarrow$ efficient optimization
- NP-completeness $\rightarrow$ hardness classification
- Approximation $\rightarrow$ near-optimal solutions
- Randomized $\rightarrow$ practical efficiency

Key Message:
Real systems use combination of all techniques

Real-World Perspective

- Robotics navigation
- AI optimization
- Cybersecurity systems
- Large-scale scheduling

Exam-Oriented Summary

- NP-hard problems are difficult
- Exact solutions are impractical
- Approximation and randomized methods are essential
- LP provides efficient optimization tools

Final Summary of Unit V

- Linear Programming and Simplex
- Geometry of feasible regions
- NP-completeness and NP-hardness
- Approximation and Randomized Algorithms